

MODULE 3

CHAPTER 1

INTRODUCTION TO SWINGS

1. Introduction to Swing

Swing did not exist in the early days of Java. Rather, it was a response to deficiencies present in Java's original GUI subsystem: the Abstract Window Toolkit. The AWT defines a basic set of controls, windows, and dialog boxes that support a usable, but limited graphical interface. One reason for the limited nature of the AWT is that it translates its various visual components into their corresponding, platform-specific equivalents, or peers. This means that the look and feel of a component is defined by the platform, not by Java. Because the AWT components use native code resources, they are referred to as heavyweight.

Swing is a part of Java Foundation Classes (JFC) and is used for building graphical user interfaces (GUIs) in Java. Unlike AWT (Abstract Window Toolkit), which uses platform-dependent components, Swing provides a rich set of GUI components that are lightweight, platform-independent, and more flexible. Swing has been the standard GUI toolkit in Java for over two decades.

2. Swing is built on the AWT

Swing eliminates a number of the limitations inherent in the AWT, Swing does not replace it. Instead, Swing is built on the foundation of the AWT. This is why the AWT is still a crucial part of Java. Swing also uses the same event handling mechanism as the AWT. Therefore, a basic understanding of the AWT and of event handling is required to use Swing.

AWT components are **heavyweight**, meaning they rely on native OS components. This creates issues:

- Appearance and behavior can vary across platforms.
- Limited customization.
- Native resource dependency.

Swing Advantages:

- **Lightweight components:** Pure Java components that do not depend on native peers.
 - **Consistent Look and Feel** across platforms.
 - **Customizable:** Developers can change component behavior and appearance easily.
-

3. Key Features of Swing

a. Swing Components are Lightweight

- Swing components are lightweight. This means that they are written entirely in Java and do not map directly to platform-specific peers.
- Thus, lightweight components are more efficient and more flexible.
- Furthermore, because lightweight components do not translate into native peers, the look and feel of each component is determined by Swing, not by the underlying operating system.
- As a result, each component will work in a consistent manner across all platforms.

b. Swing Supports a Pluggable Look and Feel (PLAF)

- It is possible to separate the look and feel of a component from the logic of the component, and this is what Swing does.
- It is possible to “plug in” a new look and feel for any given component without creating any side effects in the code that uses that component.
- Moreover, it becomes possible to define entire sets of look-and-feels that represent different GUI styles.
- To use a specific style, its look and feel is simply “plugged in.” Once this is done, all components are automatically rendered using that style.
- Swing allows changing the appearance of components at runtime.
- Examples: Metal (Java look), Nimbus, Windows, Motif.

- Developers can create **custom look and feel**.
- Provides UI consistency or platform-specific emulation.

4. Swing Architecture (Model-View-Controller)

A visual component is a composite of three distinct aspects:

- The way that the component looks when rendered on the screen
- The way that the component reacts to the user
- The state information associated with the component

The MVC architecture is successful because each piece of the design corresponds to an aspect of a component.

In MVC terminology, the model corresponds to the state information associated with the component. For example, in the case of a check box, the model contains a field that indicates if the box is checked or unchecked.

The view determines how the component is displayed on the screen, including any aspects of the view that are affected by the current state of the model.

The controller determines how the component reacts to the user.

For example, when the user clicks a check box, the controller reacts by changing the model to reflect the user's choice (checked or unchecked). This then results in the view being updated. By separating a component into a model, a view, and a controller, the specific implementation of each can be changed without affecting the other two. For instance, different view implementations can render the same component in different ways without affecting the model or the controller.

Classical MVC Model:

- **Model:** Holds the component's data/state.
- **View:** Handles component's visual representation.
- **Controller:** Handles user interactions.

Swing's Modified MVC (Model-Delegate Architecture / Separable Model Architecture):

- Combines View and Controller into a single **UI Delegate**.
- **Model** remains separate and customizable.

Example:

- JButton uses ButtonModel for state.
- UI delegate is handled by ButtonUI.

5. Swing Components and Containers

- A Swing GUI consists of two key items: components and containers.

A component is an independent visual control, such as a push button or slider. A container holds a group of components. Thus, a container is a special type of component that is designed to hold other components. Furthermore, in order for a component to be displayed, it must be held within a container. Thus, all Swing GUIs will have at least one container. Because containers are components, a container can also hold other containers. This enables Swing to define what is called a containment hierarchy, at the top of which must be a top-level container.

Components

- Swing components are derived from the JComponent class.
- JComponent supports the pluggable look and feel. JComponent inherits the AWT classes Container and Component. Thus, a Swing component is built on and compatible with an AWT component.

All standard Swing components are defined in javax.swing package. Most components extend JComponent.

Examples: Table shows the class names for Swing Components.

JApplet (Deprecated)	JButton	JCheckBox	JCheckBoxMenuItem
JColorChooser	JComboBox	JComponent	JDesktopPane
JDialog	JEditorPane	JFileChooser	JFormattedTextField
JFrame	JInternalFrame	JLabel	JLayer

JLayeredPane	JList	JMenu	JMenuBar
JMenuItem	JOptionPane	JPanel	JPasswordField
JPopupMenu	JProgressBar	JRadioButton	JRadioButtonMenuItem
JRootPane	JScrollBar	JScrollPane	JSeparator
JSlider	JSpinner	JSplitPane	JTabbedPane
JTable	JTextArea	JTextField	JTextPane
JToggleButton	JToolBar	JToolTip	JTree
JViewport	JWindow		

Containers

Used to group and manage components.

- Top-Level Containers:** JFrame, JDialog, JApplet (deprecated), JWindow
 A top-level container must be at the top of a containment hierarchy. A top-level container is not contained within any other container. Furthermore, every containment hierarchy must begin with a top-level container. The one most commonly used for applications is JFrame. In the past, the one used for applets was JApplet.
- Lightweight Containers:** JPanel, JScrollPane, JSplitPane
 The second type of containers supported by Swing are lightweight containers. Lightweight containers do inherit JComponent. An example of a lightweight container is JPanel, which is a general-purpose container. Lightweight containers are often used to organize and manage groups of related components because a lightweight container can be contained within another container. Thus, you can use lightweight containers such as JPanel to create subgroups of related controls that are contained within an outer container.

Component Hierarchy:

- Top-level containers manage:
 Each top-level container defines a set of panes. At the top of the hierarchy is an instance of JRootPane.
 JRootPane is a lightweight container whose purpose is to manage the other panes. It also helps manage the optional menu bar. The panes that comprise the root pane are called the glass pane, the content pane, and the layered pane.
 The glass pane is the top-level pane. It sits above and completely covers all other panes. By default, it is a transparent instance of JPanel. The glass pane enables you to manage mouse events that affect the entire container (rather than an individual control) or to paint over any other component.
 The layered pane is an instance of JLayeredPane. The layered pane allows components to be given a depth value.

- **JRootPane** → contains:
 - Glass Pane
 - Layered Pane
 - **Content Pane** (where components are added)

6. Swing Packages

Swing is large and includes many sub-packages:

javax.swing	javax.swing.plaf.basic	javax.swing.text
javax.swing.border	javax.swing.plaf.metal	javax.swing.text.html
javax.swing.colorchooser	javax.swing.plaf.multi	javax.swing.text.html.parser
javax.swing.event	javax.swing.plaf.nimbus	javax.swing.text.rtf
javax.swing.filechooser	javax.swing.plaf.synth	javax.swing.tree
javax.swing.plaf	javax.swing.table	javax.swing.undo

- javax.swing – Base package
- javax.swing.event – Event handling
- javax.swing.table, javax.swing.tree – Advanced components
- javax.swing.plaf – Look and Feel support

Since Java 9, all Swing packages are included in the java.desktop module.

7. A Simple Swing Program

// A simple Swing application.

```
import javax.swing.*;
```

```
class SwingDemo {
SwingDemo() {
```

```
// Create a new JFrame container.
```

```
JFrame jfrm = new JFrame("A Simple Swing Application");
```

```
// Give the frame an initial size.
```

```
jfrm.setSize(275, 100);
```

```
// Terminate the program when the user closes the application.
```

```
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
// Create a text-based label.
```

```
JLabel jlab = new JLabel(" Swing means powerful GUIs.");
```

```
// Add the label to the content pane.
```

```
jfrm.add(jlab);
```

```
// Display the frame.
```

```
jfrm.setVisible(true);
```

```
}
```

```
public static void main(String[] args) {
```

```
// Create the frame on the event dispatching thread.
```

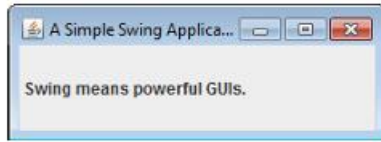
```
SwingUtilities.invokeLater(new Runnable() {
```

Prof. Keerthishree B T , Dept of CSE, AIT, Chikkamagaluru

```

public void run() {
    new SwingDemo();
}
});
}
}

```



The window produced by the **SwingDemo** program

Key Concepts:

- JFrame: Creates a main window.
- JLabel: Displays static text.
- GUI created on **Event Dispatch Thread** using `SwingUtilities.invokeLater()`.
- `javax.swing` defines classes that implement labels, buttons, text controls, and menus.
- It will be included in all programs that use Swing. Next, the program declares the `SwingDemo` class and a constructor for that class. The constructor is where most of the action of the program occurs.
- It begins by creating a `JFrame`, using this line of code: `JFrame jfrm = new JFrame("A Simple Swing Application");` This creates a container called `jfrm` that defines a rectangular window complete with a title bar; close, minimize, maximize, and restore buttons; and a system menu. Thus, it creates a standard, top-level window.
- The title of the window is passed to the constructor.
- Next, the window is sized using this statement: `jfrm.setSize(275, 100);`
- The `setSize()` method (which is inherited by `JFrame` from the AWT class `Component`) sets the dimensions of the window, which are specified in pixels. Its general form is shown here: `void setSize(int width, int height)`
- In this example, the width of the window is set to 275 and the height is set to 100.
- To terminate entire application when its top-level window is closed. There are a couple of ways to achieve this. The easiest way is to call `setDefaultCloseOperation()`.
- Using this line of code:
`jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`
- The value passed in what determines what happens when the window is closed. There are several other options in addition to `JFrame.EXIT_ON_CLOSE`. They are shown here:
`DISPOSE_ON_CLOSE`
`HIDE_ON_CLOSE`
`DO_NOTHING_ON_CLOSE`
- The next line of code creates a Swing `JLabel` component:
`JLabel jlab = new JLabel(" Swing means powerful GUIs.")` ,It simply displays information, which can consist of text, an icon, or a combination of the two.
- The next line of code adds the label to the content pane of the frame by using: `jfrm.add(jlab);`
- The last statement in the `SwingDemo` constructor causes the window to become visible: `jfrm.setVisible(true);`
- The `setVisible()` method is inherited from the AWT `Component` class. If its argument is `true`, the window will be displayed. Otherwise, it will be hidden.
- Inside `main()`, a `SwingDemo` object is created, which causes the window and the label to be displayed. Notice that the `SwingDemo` constructor is invoked using these lines of code:
`SwingUtilities.invokeLater(new Runnable() {`
`public void run() {`

- ```
 new SwingDemo();
 } });
```
- This sequence causes a SwingDemo object to be created on the event dispatching thread rather than on the main thread of the application.
- 

## 8. Event Handling in Swing

Swing uses **Delegation Event Model** for handling events.

- Components like buttons generate events.
- Events are handled by **listeners** like ActionListener.

### Example:

//Handle an event in a swing program

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
import javax.swing.*;
```

```
class EventDemo {
```

```
 JLabel jlab;
```

```
 EventDemo() {
```

```
 //create a new JFrame container
```

```
 JFrame jfrm = new JFrame("An Event Example");
```

```
 //jfrm.setLayout(new FlowLayout());
```

```
 jfrm.setLayout(new FlowLayout());
```

```
//Give the frame an initial size
```

```
 jfrm.setSize(220, 90);
```

```
 // Terminate the program when the user closes the application.
```

```
 jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
// Make two buttons.
```

```
 JButton jbtnAlpha = new JButton("Alpha");
```

```
 JButton jbtnBeta = new JButton("Beta");
```

```
// Add action listener for Alpha.
```

```
 jbtnAlpha.addActionListener(new ActionListener() {
```

```
 public void actionPerformed(ActionEvent ae) {
```

```
 jlab.setText("Alpha was pressed.");
```

```
 }
```

```
 });
```

```
// Add action listener for Beta.
```

```
 jbtnBeta.addActionListener(new ActionListener() {
```

```
 public void actionPerformed(ActionEvent ae) {
```

```
 jlab.setText("Beta was pressed.");
```

```
 }
```

```
 });
```

```
// Add the buttons to the content pane.
```

```
 jfrm.add(jbtnAlpha);
```

```
 jfrm.add(jbtnBeta);
```

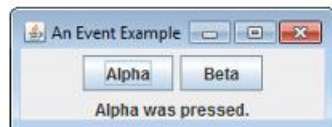
```
// Create a text-based label.
jlab = new JLabel("Press a button.");

// Add the label to the content pane.
jfrm.add(jlab);

// Display the frame. jfrm.setVisible(true);
}

public static void main(String[] args) {

 // Create the frame on the event dispatching thread.
 SwingUtilities.invokeLater(new Runnable() {
 public void run() {
 new EventDemo();
 }
 });
}
}
```



Output from the **EventDemo** program

- The EventDemo constructor begins by creating a JFrame called jfrm. It then sets the layout manager for the content pane of jfrm to FlowLayout.
- By default, the content pane uses BorderLayout as its layout manager. However, for this example, FlowLayout is more convenient.
- After setting the size and default close operation, EventDemo( ) creates two push buttons, as shown here:  
 JButton jbtnAlpha = new JButton("Alpha");  
 JButton jbtnBeta = new JButton("Beta");
- The first button will contain the text "Alpha" and the second will contain the text "Beta". Swing push buttons are instances of JButton.
- When a push button is pressed, it generates an ActionEvent. Thus, JButton provides the addActionListener( ) method, which is used to add an action listener
- ActionListener interface defines only one method: actionPerformed( ). It is shown again here :  
 void actionPerformed(ActionEvent ae)
- This method is called when a button is pressed. In other words, it is the event handler that is called when a button press event has occurred
- The buttons are added to the content pane of jfrm: jfrm.add(jbtnAlpha); jfrm.add(jbtnBeta);

## 9. Painting in Swing

### Custom Painting:

To draw directly on a component, override paintComponent(Graphics g).

- Always call super.paintComponent(g) first.
- Use drawing methods like drawLine, drawRect, etc.

### Steps:

1. Create a subclass of JPanel.
2. Override paintComponent().
3. Call repaint() to trigger drawing.

**Compute the printable Area:**

Compute the Paintable Area When drawing to the surface of a component, restrict the output to the area that is inside the border. Although Swing automatically clips any output that will exceed the boundaries of a component, it is still possible to paint into the border, which will then get overwritten when the border is drawn.

To avoid this, compute the paintable area of the component. This is the area defined by the current size of the component minus the space used by the border. Therefore, before we paint to a component, obtain the width of the border and then adjust the drawing accordingly. To obtain the border width, call `getInsets()`, shown here:

`Insets insets = getInsets();`

The inset values can be obtained by using these fields: `int top`; `int bottom`; `int left`; `int right`.

These values are then used to compute the drawing area given the width and the height of the component using `getWidth()` and `getHeight()`

**Example:**

// Paint lines to a panel.

`import java.awt.*;`

`import java.awt.event.*;`

`import javax.swing.*;`

`import java.util.*;`

// This class extends JPanel. It overrides the `paintComponent()` method so that random lines are plotted in the panel.

`class PaintPanel extends JPanel {`

`Insets insets; // holds the panel's insets`

`Random rand; // used to generate random numbers`

`// Construct a panel.`

`PaintPanel() {`

`// Put a border around the panel.`

`setBorder(`

`BorderFactory.createLineBorder(Color.RED, 5));`

`rand = new Random();`

`}`

`// Override the paintComponent() method.`

`protected void paintComponent(Graphics g) {`

`// Always call the superclass method first.`

`super.paintComponent(g);`

`int x, y, x2, y2;`

`// Get the height and width of the component.`

`int height = getHeight();`

`int width = getWidth();`

`// Get the insets.`

`insets = getInsets();`



```

// Draw ten lines whose endpoints are randomly generated.
for(int i=0; i < 10; i++) {
// Obtain random coordinates that define the endpoints of each line.
x = rand.nextInt(width-ins.left);
y = rand.nextInt(height-ins.bottom);
x2 = rand.nextInt(width-ins.left);
y2 = rand.nextInt(height-ins.bottom);

// Draw the line.
g.drawLine(x, y, x2, y2);
} } }

// Demonstrate painting directly onto a panel.
class PaintDemo {
 JLabel jlab;
 PaintPanel pp;
 PaintDemo() {

// Create a new JFrame container.
 JFrame jfrm = new JFrame("Paint Demo");

// Give the frame an initial size.
 jfrm.setSize(200, 150);
// Terminate the program when the user closes the application.
 jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

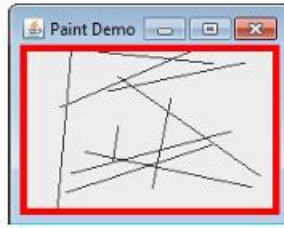
// Create the panel that will be painted.
 pp = new PaintPanel();

// Add the panel to the content pane. Because the default border layout is used, the panel will
// automatically be sized to fit the center region.

 jfrm.add(pp);
// Display the frame.
 jfrm.setVisible(true);
 }

 public static void main(String[] args) {
// Create the frame on the event dispatching thread.
 SwingUtilities.invokeLater(new Runnable() {
 public void run() {
 new PaintDemo();
 }
 });
 }
}

```



Sample output from the **PaintPanel** program

- The PaintPanel class extends JPanel. JPanel is one of Swing's lightweight containers, which means that it is a component that can be added to the content pane of a JFrame.
- To handle painting, PaintPanel overrides the paintComponent( ) method. This enables PaintPanel to write directly to the surface of the component when painting takes place.
- Border is the Swing interface that encapsulates a border. You can obtain a border by calling one of the factory methods defined by the BorderFactory class.
- The PaintDemo class creates a PaintPanel and then adds the panel to the content pane. When the application is first displayed, the overridden paintComponent( ) method is called, and the lines are drawn.
- Each time you resize or hide and restore the window, a new set of lines are drawn. In all cases, the lines fall within the paintable area

---

## Conclusion

Swing provides a rich and powerful toolkit for Java GUI programming. With its lightweight components, MVC architecture, event-driven model, and support for custom painting, Swing remains a vital part of Java development. Understanding the structure and principles behind Swing is essential for building professional and scalable desktop applications.

---

**MODULE 3**  
**CHAPTER 2**  
**EXPLORING SWING**

---

**Exploring Swing**

This chapter continues the discussion of Swing by presenting an overview of several Swing components, such as buttons, check boxes, trees, and tables.

**Components Covered:**

- JButton
- JCheckBox
- JComboBox
- JLabel
- JList
- JRadioButton
- JScrollPane
- JTabbedPane
- JTable
- JTextField
- JToggleButton
- JTree

These components are lightweight and derived from **JComponent**.

**JLabel and ImageIcon :**

- JLabel is Swing's easiest-to-use component. It creates a label.
- JLabel can be used to display text and/or an icon. It is a passive component in that it does not respond to user input

**JLabel Constructors:**

JLabel(Icon icon)

JLabel(String str)

JLabel(String str, Icon icon, int align)

Here, str and icon are the text and icon used for the label. The align argument specifies the horizontal alignment of the text and/or icon within the dimensions of the label. It must be one of the following values: LEFT, RIGHT, CENTER, LEADING, or TRAILING. These constants are defined in the SwingConstants interface

**Important Methods:**

The easiest way to obtain an icon is to use the ImageIcon class. ImageIcon implements Icon and encapsulates an image. Thus, an object of type ImageIcon can be passed as an argument to the Icon parameter of JLabel's constructor. There are several ways to provide the image, including reading it from a file or downloading it from a URL.

The icon and text associated with the label can be obtained by the following methods:

- Icon getIcon()
- String getText()

The icon and text associated with a label can be set by these methods:

- void setIcon(Icon icon)
- void setText(String str)

**Example Program:**

```
import java.awt.*;
import javax.swing.*;
```

```

public class JLabelDemo {
 public JLabelDemo() {
 // Set up the JFrame.
 JFrame jfrm = new JFrame("JLabelDemo");
 jfrm.setLayout(new FlowLayout());
 jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 jfrm.setSize(260, 210);

 // Create an icon.
 ImageIcon ii = new ImageIcon("hourglass.png");

 // Create a label.
 JLabel jl = new JLabel("Hourglass", ii, JLabel.CENTER);

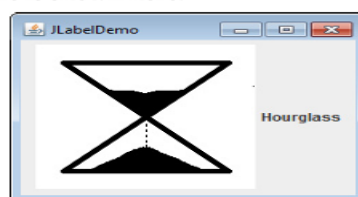
 // Add the label to the content pane.
 jfrm.add(jl);

 // Display the frame.
 jfrm.setVisible(true);
 }

 public static void main(String[] args) {
 SwingUtilities.invokeLater(new Runnable() {
 public void run() {
 new JLabelDemo();
 }
 });
 }
}

```

Output from the label example is shown here:



### **JTextField**

JTextField is the simplest Swing text component. It is also probably its most widely used text component. JTextField allows you to edit one line of text. It is derived from JTextComponent, which provides the basic functionality common to Swing text components.

#### **JTextField Constructors:**

```

JTextField(int cols)
JTextField(String str, int cols)
JTextField(String str)

```

Here, str is the string to be initially presented, and cols is the number of columns in the text field. If no string is specified, the text field is initially empty. If the number of columns is not specified, the text field is sized to fit the specified string.

JTextField generates events in response to user interaction. For example, an `ActionEvent` is fired when the user presses enter. A `CaretEvent` is fired each time the caret (i.e., the cursor) changes position.

**Example Program:**

```
// Demonstrate JTextField.
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JTextFieldDemo {
 public JTextFieldDemo() {

// Set up the JFrame.
 JFrame jfrm = new JFrame("JTextFieldDemo");
 jfrm.setLayout(new FlowLayout());
 jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 jfrm.setSize(260, 120);

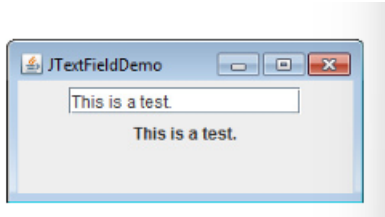
// Add a text field to content pane.
 JTextField jtf = new JTextField(15);
 jfrm.add(jtf);

// Add a label.
 JLabel jlab = new JLabel();
 jfrm.add(jlab);

// Handle action events.
 jtf.addActionListener(new ActionListener() {
 public void actionPerformed(ActionEvent ae) {
 jlab.setText(jtf.getText());
 }
 });

// Display the frame.
 jfrm.setVisible(true);
 }

 public static void main(String[] args) {
// Create the frame on the event dispatching thread.
 SwingUtilities.invokeLater(new Runnable() {
 public void run() {
 new JTextFieldDemo();
 }
 });
 }
}
```



## **The Swing Buttons**

Swing defines four types of buttons: JButton, JToggleButton, JCheckBox, and JRadioButton. All are subclasses of the AbstractButton class, which extends JComponent.

### **AbstractButton Methods:**

- void setDisabledIcon(Icon di)
- void setPressedIcon(Icon pi)
- void setSelectedIcon(Icon si)
- void setRolloverIcon(Icon ri)

The text associated with a button can be read and written via the following methods:

- String getText()
- void setText(String str)

### **JButton :**

The JButton class provides the functionality of a push button. JButton allows an icon, a string, or both to be associated with the push button. Three of its constructors are shown here:

### **JButton Constructors:**

JButton(Icon icon)

JButton(String str)

JButton(String str, Icon icon)

When the button is pressed, an ActionEvent is generated. Using the ActionEvent object passed to the actionPerformed() method of the registered ActionListener, you can obtain the action command string associated with the button. By default, this is the string displayed inside the button. However, you can set the action command by calling setActionCommand() on the button. Obtain the action command by calling getActionCommand() on the event object.

### **Example Program:**

The following demonstrates an icon-based button. It displays four push buttons and a label. Each button displays an icon that represents a timepiece. When a button is pressed, the name of that timepiece is displayed in the label.

```
// Demonstrate an icon-based JButton.
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JButtonDemo implements ActionListener {
 JLabel jlab;

 public JButtonDemo() {
 // Set up the JFrame.
 JFrame jfrm = new JFrame("JButtonDemo");
```

```
jfrm.setLayout(new FlowLayout());
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
jfrm.setSize(500, 450);

// Add buttons to content pane.
ImageIcon hourglass = new ImageIcon("hourglass.png");
JButton jb = new JButton(hourglass);
jb.setActionCommand("Hourglass");
jb.addActionListener(this);
jfrm.add(jb);

ImageIcon analog = new ImageIcon("analog.png");
jb = new JButton(analog);
jb.setActionCommand("Analog Clock");
jb.addActionListener(this);
jfrm.add(jb);

ImageIcon digital = new ImageIcon("digital.png");
jb = new JButton(digital);
jb.setActionCommand("Digital Clock");
jb.addActionListener(this);
jfrm.add(jb);

ImageIcon stopwatch = new ImageIcon("stopwatch.png");
jb = new JButton(stopwatch);
jb.setActionCommand("Stopwatch");
jb.addActionListener(this);
jfrm.add(jb);

// Create and add the label to content pane.
JLabel jlab = new JLabel("Choose a Timepiece");
jfrm.add(jlab);

// Display the frame.
jfrm.setVisible(true);
}

// Handle button events.
public void actionPerformed(ActionEvent ae) {
 jlab.setText("You selected " + ae.getActionCommand());
}

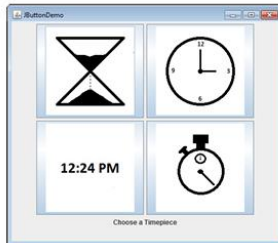
public static void main(String[] args) {
// Create the frame on the event dispatching thread.
SwingUtilities.invokeLater(new Runnable() {
 public void run() {
```

```

 new JButtonDemo();
 }
 });
}
}

```

Output from the button example is shown here:



### JToggleButton

A useful variation on the push button is called a toggle button. A toggle button looks just like a push button, but it acts differently because it has two states: pushed and released. That is, when you press a toggle button, it stays pressed rather than popping back up as a regular push button does. When you press the toggle button a second time, it releases (pops up). Therefore, each time a toggle button is pushed, it toggles between its two states.

Toggle buttons are objects of the JToggleButton class. JToggleButton implements AbstractButton.

#### JToggleButton Constructor:

JToggleButton(String str)

#### Example Program:

```

// Demonstrate JToggleButton.
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JToggleButtonDemo {
 public JToggleButtonDemo() {
 // Set up the JFrame.
 JFrame jfrm = new JFrame("JToggleButtonDemo");
 jfrm.setLayout(new FlowLayout());
 jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 jfrm.setSize(200, 100);

 // Create a label.
 JLabel jlab = new JLabel("Button is off.");

 // Make a toggle button
 JToggleButton jtbn = new JToggleButton("On/Off");

 // Add an item listener for the toggle button.
 jtbn.addItemListener(new ItemListener() {

```



```

 public void itemStateChanged(ItemEvent ie) {
 if(jtbn.isSelected())
 jlab.setText("Button is on.");
 else
 jlab.setText("Button is off.");
 }
});

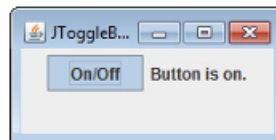
// Add the toggle button and label to the content pane.
jfrm.add(jtbn);
jfrm.add(jlab);

// Display the frame.
jfrm.setVisible(true);
}

public static void main(String[] args) {
// Create the frame on the event dispatching thread.
SwingUtilities.invokeLater(new Runnable() {
 public void run() {
 new JToggleButtonDemo();
 }
});
}
}

```

The output from the toggle button example is shown here:



### JCheckBox

The JCheckBox class provides the functionality of a check box. Its immediate superclass is JToggleButton, which provides support for two-state buttons.

JCheckBox defines several constructors.

The one used here is JCheckBox(String str)

It creates a check box that has the text specified by str as a label.

When the user selects or deselects a check box, an ItemEvent is generated. You can obtain a reference to the JCheckBox that generated the event by calling getItem( ) on the ItemEvent passed to the itemStateChanged( ) method defined by ItemListener. The easiest way to determine the selected state of a check box is to call isSelected( ) on the JCheckBox instance.

#### Example Program:

// Demonstrate JCheckbox.

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

```

```
public class JCheckBoxDemo implements ItemListener {
 JLabel jlab;

 public JCheckBoxDemo() {
 // Set up the JFrame.
 JFrame jfrm = new JFrame("JCheckBoxDemo");
 jfrm.setLayout(new FlowLayout());
 jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 jfrm.setSize(250, 100);

 // Add check boxes to the content pane.
 JCheckBox cb = new JCheckBox("C");
 cb.addItemListener(this);
 jfrm.add(cb);

 cb = new JCheckBox("C++");
 cb.addItemListener(this);
 jfrm.add(cb);

 cb = new JCheckBox("Java");
 cb.addItemListener(this);
 jfrm.add(cb);

 cb = new JCheckBox("Perl");
 cb.addItemListener(this);
 jfrm.add(cb);

 // Create the label and add it to the content pane.
 jlab = new JLabel("Select languages");
 jfrm.add(jlab);
 // Display the frame.
 jfrm.setVisible(true);
 }

 // Handle item events for the check boxes.
 public void itemStateChanged(ItemEvent ie) {
 JCheckBox cb = (JCheckBox)ie.getItem();

 if(cb.isSelected())
 jlab.setText(cb.getText() + " is selected");
 else
 jlab.setText(cb.getText() + " is cleared");
 }

 public static void main(String[] args) {
 // Create the frame on the event dispatching thread.
 SwingUtilities.invokeLater(new Runnable() {
 public void run() {
```

```

 new JCheckBoxDemo();
 }
 });
}
}

```



### JRadioButton

Radio buttons are a group of mutually exclusive buttons, in which only one button can be selected at any one time. They are supported by the `JRadioButton` class, which extends `JToggleButton`. `JRadioButton` provides several constructors.

The one used in the example is shown here: `JRadioButton(String str)`

Here, `str` is the label for the button.

A button group is created by the `ButtonGroup` class

A `JRadioButton` generates action events, item events, and change events each time the button selection changes. Most often, it is the action event that is handled, which means that you will normally implement the `ActionListener` interface. The only method defined by `ActionListener` is `actionPerformed()`.

The following example illustrates how to use radio buttons. Three radio buttons are created. The buttons are then added to a button group. As explained, this is necessary to cause their mutually exclusive behavior. Pressing a radio button generates an action event, which is handled by `actionPerformed()`. Within that handler, the `getActionCommand()` method gets the text that is associated with the radio button and uses it to set the text within a label

#### Example Program:

```

// Demonstrate JRadioButton
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JRadioButtonDemo implements ActionListener {
 JLabel jlab;

 public JRadioButtonDemo() {
 // Set up the JFrame.
 JFrame jfrm = new JFrame("JRadioButtonDemo");
 jfrm.setLayout(new FlowLayout());
 jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 jfrm.setSize(250, 100);

 // Create radio buttons and add them to content pane.
 JRadioButton b1 = new JRadioButton("A");
 b1.addActionListener(this);
 jfrm.add(b1);
 }
}

```

```
JRadioButton b2 = new JRadioButton("B");
b2.addActionListener(this);
jfrm.add(b2);

JRadioButton b3 = new JRadioButton("C");
b3.addActionListener(this);
jfrm.add(b3);

// Define a button group.
ButtonGroup bg = new ButtonGroup();
bg.add(b1);
bg.add(b2);
bg.add(b3);

// Create a label and add it to the content pane.
JLabel jlab = new JLabel("Select One");
jfrm.add(jlab);

// Display the frame.
jfrm.setVisible(true);
}

// Handle button selection.
public void actionPerformed(ActionEvent ae) {
 jlab.setText("You selected " + ae.getActionCommand());
}

public static void main(String[] args) {
// Create the frame on the event dispatching thread.
SwingUtilities.invokeLater(new Runnable() {
 public void run() {
 new JRadioButtonDemo();
 }
});
}
}
```

